

C++ object oriented programming

- ① object oriented programming refers to languages that use objects in programming.
- ② object oriented programming is about creating objects that contain both data and functions.

Advantages of Object Oriented Programming

- * faster and easier to execute.
 - * provides a clear structure for the programs.
 - * makes the code easier to maintain, modify and debug.
 - * usefull to create reusable applications with less code and shorter development time.
- ③ classes and objects are two main aspects of object oriented programming.

Class

- * A class contain data members and member functions, that can be accessed and used by creating an instance of that class.
 - * A class represents a set of properties or methods that are common for all objects of one type.
- ④ Simply; a class is a template / blueprint for an object.

Object

- * basic unit of object oriented programming. When a class is defined, no memory is allocated until an instance is created.
- ⑤ Object is an instance of a class.

A when an individual object is created, they inherit all the variables and functions ~~to~~ from it's class.

- * every object in C++ has attributes and methods. They are variables and classes that belongs to the class respectively.
- * 'class' is another user-defined data type!

Create classes

- ① To create a class, we use class keyword.

E.x.

```
class myClass {  
    public:   
        int myNum;  
        string myString;  
};
```

Annotations:
- public: Access Specifier
- { int myNum; string myString; } Attributes

- ② To create an object, we just put an object name followed by class name. To access the attributes of a class we use dot syntax.

E.x.

```
myClass myObj;  
myObj.myNum = 15;  
myObj.myString = "Hello";
```

Annotations:
- myClass myObj; Create an object
- { myObj.myNum = 15; myObj.myString = "Hello"; } Access attributes and set values

- * We can create class methods in two ways;
 - * Inside class definition
 - * Outside class definition

Ex. ① // define a function inside the class

```
class myClass {  
    public:  
        void myMethod() {  
            cout << "Hello";  
        }  
};
```

Annotation: { void myMethod() { ... } method inside the class

```
int main() {  
    myClass myObj;  
    myObj.myMethod();  
    return 0;  
}
```

Annotations:
- myClass myObj; create an object
- myObj.myMethod(); call the method

To declare a method outside the class definition;
first, we have to declare that method inside the class.
then, we have to define it outside the class.

* In this case we specify the name of class, followed by scope resolution operator (::), then the name of the function.

E.x. ②

```
class myClass {  
    public:  
        void myMethod();  
};  
  
void myClass::myMethod() {  
    cout << "Hello";  
}  
  
int main() {  
    myClass myObj;  
    myObj.myMethod();  
    return 0;  
}
```

Note: we can also use parameters for these class methods as usual way.

Constructors

- * A constructor is a special method that is automatically called when an object is created.
- * Constructor has the same name as the class and it followed by parentheses ()

E.x. class myClass {

public:

```
myClass() {  
    cout << "Hello";  
}
```

};

← Constructor of
the class
(same name as the class)

Note: The constructor has the same name as the class, it is always has the 'public' as the access specifier.

★ Constructors can also have parameters, that useful for set initial values for attributes.

E.x. class Car {

public:

string brand;

string model;

int year;

```
Car (string x, string y, int z) {
```

brand = x;

model = y;

year = z;

}

};

← Constructor
with
parameters

int main() {

Car obj1("BMW", "X5", 1999);

Car obj2("Ford", "Mustang", 1969);

cout << obj1.brand << endl;

cout << obj2.year << endl;

return 0;

}

← Create car objects
and call the
constructor with
different values.

As class methods, we can also define a constructor outside the class.

```
E.x. class Car {
    public:
        string brand;
        string model;
        int year;
        Car(string x, string y, int z);
};

Car::Car(string x, string y, int z) {
    brand = x;
    model = y;
    year = z;
}
```

Access Specifiers

- ⊛ These are the things that describe how the members of a class can be accessed.
- ⊛ There are three access specifiers in C++ ;
 - i.) public → members are accessible from outside the class.
 - ii.) private → members cannot be accessed from outside the class.
 - iii.) protected → members cannot be accessed from outside the class, but they can be accessed in inherited classes.

```
E.x. class MyClass {
    public:
        int x;
    private:
        int y;
};

int main() {
    MyClass myObj;
    myObj.y = 50;
    return 0;
}
```

private attribute

// Error
(private attributes can't access outside the class)

Note: It is possible to access private members of a class using a public method inside the same class.

Ⓐ By default; all members of a class are 'private'

E.x. class myClass {
 int x;
 int y;
};

} ← private attributes

Object Oriented Principles

Ⓐ Object oriented Principles mainly include 4 pillars that together make the OOP a very powerful concept.

- 1.) Abstraction
- 2.) Encapsulation
- 3.) Inheritance
- 4.) Polymorphism

Encapsulation

Ⓐ Meaning is; 'make sure that "sensitive" data is hidden from users.'

Ⓐ For this we declare all the sensitive data contained class members as 'private'.

Ⓐ If we want to give a chance to read or modify the value of a private member, we can provide public 'get' and 'set' methods.

E.x. class Employee {
 private:
 int salary;
 public:
 void setSalary(int s) {
 salary = s;
 }
 int getSalary() {
 return salary;
 }
};

} ← Setter method
(set the value of a private member)

} ← Getter method
(return a private member)

```

int main() {
    Employee myObj;
    myObj.setSalary(50000);
    cout << myObj.getSalary();
    return 0;
}

```

Note: That is a good practice to declare class attributes as 'private'. Encapsulation ensures better control of the data and increase the security of data.

Inheritance

- ① Inheritance is a feature or a process which, new classes are created from the existing classes.
- ② We can group 'inheritance' into two categories;
 - * derived class (child) → the class that inherits from another class
 - * base class (parent) → the class being inherited from
- ③ To inherit from a class, use ":" symbol.

Single inheritance

- ① Single inheritance is which the derived class inherits the single base class either public, private or protected.
- ② In single inheritance, the derived class uses the features or members of the single base class.

```

E.x. class A {
    public:
        int k = 1000;
        float salary = 80000;
};

class B : class A {
    public:
        float bonus = 8000;
        void ts() {
            cout << "Total salary : " << (salary + bonus);
        }
};

int main() {
    B b1;
    cout << "Salary : " << b1.salary << endl;
    b1.ts();

    return 0;
}

```

⇓ output

```

Salary : 80000
Total Salary : 88000

```

Multiple inheritance

- ⊛ Multiple inheritance is one in which the derived class acquires two or more base classes.
- ⊛ In multiple inheritance; the derived is allowed to use the joint features of the inherited base classes.

E.x.

```
class A {  
    protected:  
        int a;  
    public:  
        void get_a(int n) {  
            a=n;  
        }  
};
```

```
class B {  
    protected:  
        int b;  
    public:  
        void get_b(int n) {  
            b=n;  
        }  
};
```

```
class C : public A, public B {  
    public:  
        void display() {  
            cout << "The value of a is:" << a << endl;  
            cout << "The value of b is:" << b << endl;  
            cout << "Product of a and b is:" << a*b;  
        }  
};
```

```
int main() {  
    C c;  
    c.get_a(10);  
    c.get_b(20);  
    c.display();  
  
    return 0;  
}
```

Differences between single inheritance and multiple inheritance

Single Inheritance	Multiple Inheritance
* Single inheritance is which the derived class inherits the single base class.	* Multiple inheritance is which the derived class acquires two or more base classes.
* derived class uses the features of the single base class.	* derived class uses the joint features of the inherited base classes.
* requires a small run time due to less overhead.	* requires comparatively more run time due to more overhead.
* lot close to specialization.	* lot close to generalization.
* Comparatively simple to implement.	* Complex to implement.
* Can be implemented in any programming language.	* Some languages (C#, Java) not support.

Multilevel inheritance

- ⊛ A class can be derived from one class, which is already derived from another class.

E.x.

```
class myClass {  
    public:  
        void myFunction() {  
            cout << "Some content in parent class.";  
        }  
};  
  
class myChild : public myClass {  
};
```

```
class myGrandChild : public myChild {  
};
```

```
int main () {  
    myGrandChild myObj;  
    myObj.myfunction();  
  
    return 0;  
}
```

- ★ We can use 'protected' as the access specifier and it is much similar to the 'private' access specifier.

But the main difference is, it can be accessed in the "Inherited" class.

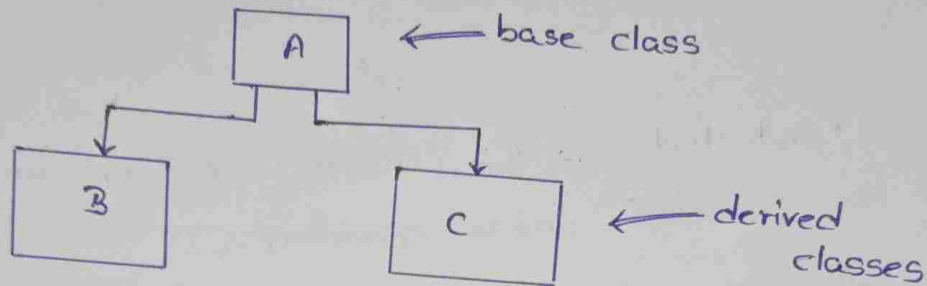
```
E.x. class Employee {  
    protected:  
        int salary;  
};
```

```
class Programmer : public Employee {  
    public:  
        int bonus;  
        void setSalary(int s) {  
            salary = s;  
        }  
        int getSalary() {  
            return salary;  
        }  
};
```

```
int main () {  
    Programmer myObj;  
    myObj.setSalary(50000);  
myObj  
    cout << myObj.getSalary() << endl;  
  
    return 0;  
}
```

Hierarchical inheritance

- ① Hierarchical inheritance is the concept, more than one sub-class inherits the property of a single base class. There is one base class and multiple derived classes.
- ② Hierarchical structures thus form a tree-like structure.



```
E.x. class A {
    public:
        void show_A() {
            cout << "class A" << endl;
        }
};

class B : public A {
    public:
        void show_B() {
            cout << "class B" << endl;
        }
};

class C : public A {
    public:
        void show_C() {
            cout << "class C" << endl;
        }
};

int main() {
    B b;
    cout << "From B:" << endl << b.show_B() << b.show_A();
    C c;
    cout << "From C:" << endl << c.show_C() << c.show_A();
}
```

Binding

refer www.scaler.com for more about 'binding'

- ⊛ Binding helps in 'linking a bridge' between a function call and its corresponding function definition.
- ⊛ Consider both function definition and function calls are stored in the memory at "separate addresses". For smooth operation, we need a technique to match the appropriate function call with its definition.
- ⊛ Process of matching a specific function call to its respective function definition is known as "binding".
- ⊛ There are two types of binding ;
 - i.) static binding
 - ii.) dynamic binding
- ⊛ Binding at compile time is known as "static binding".
- ⊛ Binding at runtime is known as "dynamic binding".

Differences between static binding and dynamic binding

Static Binding	Dynamic Binding
★ During compile time only. It is by default in C++	★ During the run time.
★ Early binding, compile-time binding.	★ Late binding, run-time binding.
★ All the information to resolve function calls is available simultaneously.	★ The compiler cannot determine all the information at compile time.
★ more faster.	★ more slower.
★ Function overloading or operator overloading in C++	★ Virtual functions in C++

Polymorphism

⊛ ~~Def~~ Polymorphism is an idea that an entity in code such as a variable, function or an object can have more than one form.

⊛ This allows us to perform a single action in different ways.

```
E.x. class Animal {  
    public:  
        void animalSound() {  
            cout << "The animal makes a sound \n";  
        }  
};
```

```
class Pig : public Animal {  
    public:  
        void animalSound() {  
            cout << "The pig says : wee wee \n";  
        }  
};
```

```
class Dog : public Animal {  
    public:  
        void animalSound() {  
            cout << "The dog says : bow wow \n";  
        }  
};
```

```
int main() {  
    Animal myAnimal;  
    Pig myPig;  
    Dog myDog;  
  
    myAnimal.animalSound();  
    myPig.animalSound();  
    myDog.animalSound();  
  
    return 0;  
}
```

★ Pig and Dog objects created and override the animalSound() method.

Files in C++

- ① `<fstream>` library supports to work with files.
- ② There are three classes included in fstream library.

`ofstream` → creates and writes to files

`ifstream` → Reads from files

`fstream` → A combination of `ofstream` and `ifstream`:
create, read and write to files.

- ③ To create a file use `ofstream` or `fstream` class and specify the name of the file. Then use 'insertion operator' (`<<`) to write to the file.

E.x. `#include <iostream>`
`#include <fstream>`
`using namespace std;`

`int main() {`

`ofstream MyFile("filename.txt"); // Create and opens a text file.`

`MyFile << "Here I'm writing in the file";`

`// write in the file.`

`MyFile.close();`

`return 0;`

`}`

note: It is a good practice to close the used files.

- ④ To read from a file use `ifstream` or `fstream` class and use a while loop together with `getline()` function.

```

E.x. string myText;
      ifstream myReadFile("filename.txt");
      while (getline(myReadFile, myText)) {
          cout << myText;
      }
      myReadFile.close();

```

Exceptions

- * When we executing the source code in C++, errors can be occur due to many reasons.
- * In that situation, C++ will normally stop the execution and generate an error message. In ~~techn~~ technical term;
 "C++ will throw an exception".
- * With the use of 'try-catch' blocks we can handle those exceptions.
 - try :- define a block of code to be tested for errors while it is being executed.
 - throw :- throws an exception when a problem is detected.
 we can create a custom error.
 - catch :- define a block of code to be executed, if an error occurs in 'try block'.

Note: 'try' and 'catch' keywords come in pairs

```

E.x. try {
      int age = 15;
      if (age >= 18) {
          cout << "Access granted";
      } else {
          throw (age);
      }
  }
  catch (int myNum) {
      cout << "Access denied" << endl;
      cout << "Age is : " << myNum;
  }

```

If we don't know the 'type' used in try block, we can use "three dot syntax" (...) inside the catch block.
It can handle any type of exception.

```
E.x. try {  
    throw 'a';  
}  
catch (int x) {  
    cout << "Caught " << x;  
}  
catch (...) {  
    cout << "Default Exception";  
}
```

Vectors

- * Vectors are the resizable arrays in C++. So, unlike arrays we can modify the ~~vector~~ size of the vector as we needed.
- * To use vectors, we must include the <vector> header file.
- * To create a vector, we use vector keyword and specify the type of values within angle brackets (<>).

```
E.x. vector<string> cars = {"Volvo", "BMW", "Ford"};  
for (string car : cars) { // for-each loop  
    cout << car << endl;  
}
```

- * In vectors, we can use .front() and .back() functions to access the first and last elements of a vector respectively.

```
E.x. cout << cars.front(); // "Volvo"  
      cout << cars.back(); // "Ford"
```

- * Use indexing within square brackets [] or with .at() function to access any element of a vector.

★ For adding vector elements we can use `.push-back()` function and it will add an element at the end of the vector.

E.x. `vector<string> cars = {"Volvo", "BMW", "Ford"};`
`cars.push-back("Tesla");`

// "Tesla" add to the end of the vector

★ Using `.pop-back()` function, we can remove the last element of a vector.

E.x. `cars.pop-back();`

★ `.erase()` function ~~can~~ use to remove one or more elements from a vector.

E.x. 1) `vector<int> v = {1, 2, 3, 4, 5};`

`v.erase(v.begin() + 2);`

// Remove the element at index 2

2) `v.erase(v.end() - 2);`

// Remove the element before the last element

3) `v.erase(v.begin() + 1, v.begin() + 4);`

// Remove elements in the range [1, 4)

★ Using `.size()` function, we can get the number of elements in a vector.

★ `.empty()` function used for check whether a vector is empty or not.

E.x. `cout << cars.empty();`

// Outputs 1 (if the vector is empty)

// Outputs 0 (if the vector is not empty)